

Datos del estudiante

Osvaldo Bobadilla Mino

19/06/2026

Visualizaciones interactivas con datos JSON y D3.js

Objetivos de la actividad

Una empresa de tecnología desea evaluar el desempeño de sus productos (*smartphone A*, *smartphone B* y *tablet X*) para optimizar la estrategia comercial. Se requiere desarrollar un *dashboard* interactivo que permita analizar la evolución de ventas, ingresos y precios de dichos productos durante los meses de enero y febrero. Los objetivos específicos son:

- ▶ Integrar y procesar datos en formato JSON desde una fuente externa, simulando la actualización dinámica de información.
- ▶ Desarrollar una visualización básica en tiempo real utilizando la Google Chart Library para mostrar, por ejemplo, un gráfico de columnas que represente las ventas mensuales.
- ▶ Implementar una visualización avanzada con D3.js que incluya animaciones, transiciones y manejo de eventos (*mouseover*, *click*, etc.) para analizar múltiples dimensiones (ventas, ingresos, precio y producto).
- ▶ Garantizar que ambas visualizaciones sean escalables y se adapten a diferentes dispositivos, ofreciendo una experiencia interactiva al usuario.

Desarrollo del Dashboard Interactivo de Visualización de Datos con Node.js, Google Charts y D3.js

Metodología: Metodología

El desarrollo del dashboard interactivo se realizó mediante una metodología de desarrollo incremental, donde se implementaron y probaron progresivamente los componentes necesarios para la adquisición, procesamiento, análisis y visualización de datos. El proyecto se estructuró bajo una arquitectura cliente-servidor utilizando tecnologías web, con la finalidad de que el usuario pueda interactuar de manera amigable con la consola y realizar consultas en tiempo real.

El proyecto se organizó de la siguiente manera: La carpeta **data** contiene los archivos de información en formato JSON utilizados como fuente de datos; la carpeta **public** almacena los elementos correspondientes a la interfaz web, incluyendo archivos HTML, hojas de estilo CSS y scripts JavaScript; adicionalmente, se incorporaron archivos de configuración para la ejecución del servidor mediante Node.js.

Posteriormente se implementó un servidor web utilizando **Node.js con Express**, encargado de gestionar las solicitudes realizadas desde la interfaz del usuario. Mediante un endpoint interno, la aplicación consulta la información almacenada en el archivo JSON y la entrega al navegador utilizando peticiones HTTP, permitiendo que los datos sean cargados dinámicamente sin necesidad de modificar manualmente la página.

La etapa de procesamiento de datos se desarrolló mediante JavaScript en el lado del cliente. En esta fase se realizaron operaciones de filtrado, agrupación y cálculo de indicadores, considerando variables como producto, año, mes, ventas, ingresos y precio. También se implementaron filtros interactivos que permiten al usuario seleccionar diferentes categorías de análisis, así como periodos trimestrales para comparar el comportamiento comercial en diferentes intervalos de tiempo como son años anteriores.

Para la representación gráfica de la información se utilizaron dos bibliotecas especializadas en visualización de datos:

- **Google Charts**, utilizado para la construcción de gráficos de barras y gráficos circulares, permitiendo representar la evolución histórica de ventas y la participación porcentual de cada producto.
- **D3.js (Data Driven Documents)**, empleado para desarrollar visualizaciones dinámicas como el gráfico de burbujas y el diagrama de caja. Estas gráficas permiten representar relaciones entre múltiples variables, como la relación entre ventas, ingresos y precio, además de analizar la distribución estadística de las ventas

mediante valores mínimos, máximos, el rango intercuartílico y en caso de existir también los outliers.

El diseño del dashboard se realizó buscando que cada gráfico estuviera acompañado por elementos descriptivos y tablas relacionadas para facilitar la interpretación de resultados. Se implementó una distribución responsiva mediante CSS, permitiendo adaptar la visualización a diferentes tamaños de pantalla y mantener una organización clara entre gráficos, indicadores y tablas.

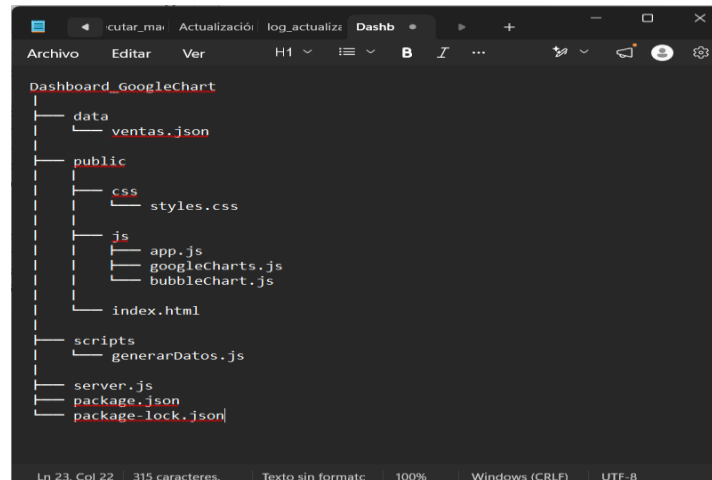
Finalmente, se añadió un botón “Actualizar” para poder refrescar la conexión a la base de datos verificando que el sistema pudiera iniciar con un histórico de información hasta febrero de 2026 y posteriormente ampliar el análisis hasta junio de 2026. Además, se validó el correcto funcionamiento de filtros, cálculos estadísticos y generación automática de visualizaciones.

Tecnologías utilizadas	
Componente	Tecnología
Lenguaje principal	JavaScript
Servidor web	Node.js + Express
Estructura web	HTML5 + CSS3
Fuente de datos	JSON
Visualización gráfica	Google Charts
Visualización interactiva	D3.js
Control de versiones	Git + GitHub
Entorno de desarrollo	Visual Studio Code

1. Creación de la estructura del proyecto

Inicialmente se creó una carpeta principal para organizar todos los componentes del dashboard: Dashboard_GoogleChart

Dentro de esta carpeta se definió una estructura modular para separar datos, archivos del cliente, scripts auxiliares y configuración del servidor:

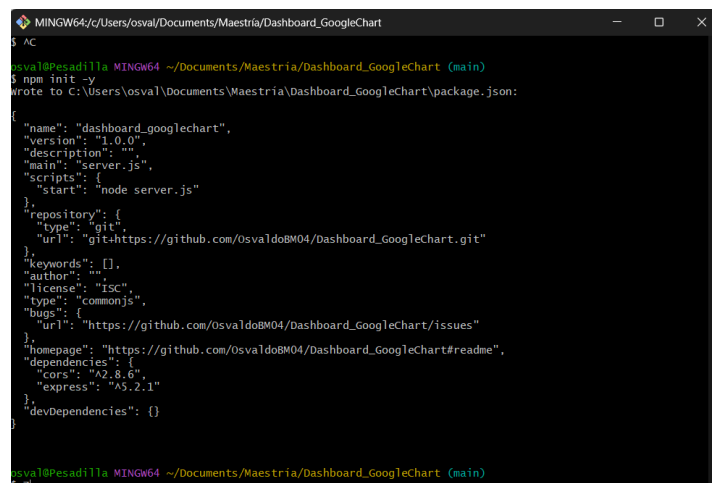


La organización permitió separar:

- **Datos:** archivos JSON con la información de ventas.
- **Interfaz:** archivos HTML, CSS y JavaScript del navegador.
- **Servidor:** encargado de entregar recursos mediante HTTP.
- **Scripts auxiliares:** generación y actualización de información.

2. Preparación del entorno Node.js

Se inicializó el proyecto utilizando Node.js para crear la configuración necesaria: `npm init` esto generó un archivo llamado `package.json` para almacenar las configuraciones y dependencias necesarias para el proyecto.



posteriormente se instaló Express, librería utilizada para crear el servidor web: `npm install express`. Esto permitió crear una aplicación capaz de servir archivos mediante HTTP.

3. Creación del archivo de datos JSON

Los datos suministrados fueron almacenados en: data/ventas.json. El objetivo fue evitar colocar los datos directamente dentro del código JavaScript, de forma que la visualización consulta el archivo externo y procesa la información dinámicamente.

4. Implementación del servidor HTTP

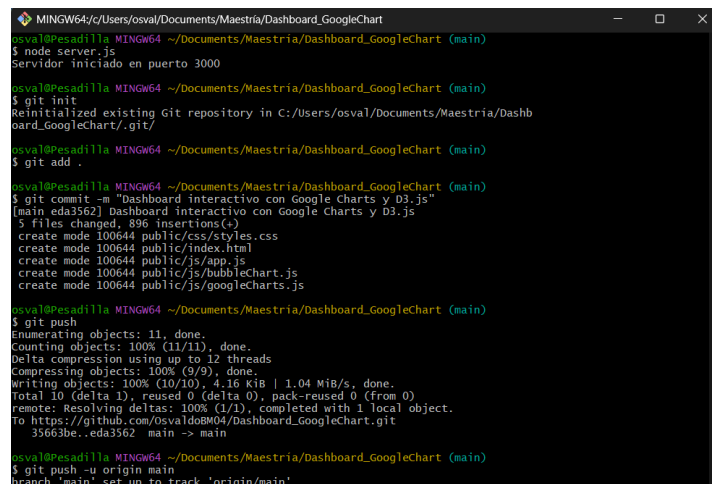
Se creó el archivo: server.js. Este archivo implementa un servidor utilizando Node.js y Express.

Su función principal es:

1. Servir los archivos del dashboard.
2. Exponer el archivo JSON mediante una ruta API.

5. Levantamiento del servidor

Para iniciar la aplicación se ejecutó: node server.js



```
MINGW64~/Users/osval/Documents/Maestria/Dashboard_GoogleChart
osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ node server.js
Servidor iniciado en puerto 3000

osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git init
Reinitialized existing Git repository in C:/Users/osval/Documents/Maestria/Dash
board_GoogleChart/.git/

osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git add .

osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git commit -m "Dashboard interactivo con Google Charts y D3.js"
[main eda3562] Dashboard interactivo con Google Charts y D3.js
5 files changed, 896 insertions(+)
create mode 100644 public/css/styles.css
create mode 100644 public/index.html
create mode 100644 public/js/app.js
create mode 100644 public/js/bubbleChart.js
create mode 100644 public/js/googleCharts.js

osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git push
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 4.16 KiB | 1.04 MiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
to https://github.com/osvaldo804/Dashboard_GoogleChart.git
35663be..eda3562 main -> main

osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git push -u origin main
branch 'main' set up to track 'origin/main'.
```

El servidor mostró: Servidor iniciado en puerto 3000. A partir de este momento el navegador dejó de cargar archivos directamente desde el sistema operativo y comenzó a solicitarlos mediante HTTP.

6. Carga de recursos mediante el servidor

Al acceder a: <http://localhost:3000> el servidor entrega: <public/index.html>. Posteriormente el navegador carga: <css/styles.css> así como funciones JavaScript: <js/app.js>, <js/googleCharts.js>, <js/bubbleChart.js>.

De esta forma desde el archivo: `public/js/app.js` se realiza la consulta mediante `fetch("/api/ventas")` llegando la petición al servidor de la siguiente manera <http://localhost:3000/api/ventas> el servidor responde enviando `data/ventas.json`

Posteriormente JavaScript transforma la información para alimentar los gráficos.

Esto permite mantener separados:

- código de visualización
- fuente de datos

evitando almacenar información fija dentro del programa.

7. Implementación de visualización con Google Charts y D3.js

Para la etapa de visualización del dashboard se integraron librerías especializadas en representación gráfica de datos, utilizando **Google Charts** y **D3.js**, con el objetivo de transformar los registros comerciales en elementos visuales interactivos que permitieran analizar tendencias, comparaciones y distribución estadística de las variables.

Inicialmente se incorporó la librería de Google Charts mediante: `<script src="https://www.gstatic.com/charts/loader.js"></script>`.

Esta herramienta permitió desarrollar un gráfico de columnas utilizando la función: `drawGoogleChart(datos)`.

La función recibe los datos procesados desde el archivo JSON y realiza una agrupación por periodo de tiempo y producto, organizando la información para comparar el comportamiento histórico de ventas.

El gráfico representa:

- **Eje X:** Periodo de tiempo (año-mes)
- **Eje Y:** Cantidad de ventas (unidades)
- **Series:** Productos tecnológicos analizados

Permitiendo comparar:

- Smartphone A

- Smartphone B
- Tablet X

durante los diferentes periodos disponibles.

Posteriormente se implementó un gráfico de pastel utilizando Google Charts, orientado al análisis de participación comercial (o bien participación porcentual), permitiendo identificar qué productos tienen mayor contribución dentro del total de ventas.

La representación permite observar:

- Producto → categoría de análisis
- Ventas → proporción dentro del total comercial

Implementación de visualizaciones con D3.js

Para generar visualizaciones más avanzadas e interactivas se incorporó la librería:

```
<script src="https://d3js.org/d3.v7.min.js"></script>
```

D3.js permitió desarrollar gráficos dinámicos mediante manipulación directa del DOM y escalas matemáticas para representar relaciones entre variables.

Se implementó un gráfico de burbujas mediante la función:

```
drawBubbleChart(datos)
```

Esta visualización permite analizar simultáneamente tres dimensiones del comportamiento comercial:

- **Producto** → color de la burbuja
- **Ventas** → posición en el eje X
- **Ingresos** → posición en el eje Y
- **Precio** → tamaño de la burbuja

Cada elemento representa un registro comercial, permitiendo observar relaciones entre volumen de ventas, generación de ingresos y precio promedio del producto.

La gráfica incorpora elementos interactivos como:

- Animaciones mediante transiciones (`transition`)
- Eventos de interacción con el usuario (`mouseover`)
- Ventanas emergentes informativas (`tooltips`)

- Eventos de selección mediante clic (`click`)

Los tooltips muestran información detallada del registro seleccionado:

- Producto
- Año
- Mes
- Cantidad de ventas
- Ingresos generados
- Precio

facilitando la exploración individual de los datos.

Implementación del análisis estadístico mediante Boxplot

Como complemento al análisis descriptivo se desarrolló un diagrama de caja utilizando D3.js mediante la función:

```
drawBoxplot(datos)
```

Esta visualización permite analizar la distribución estadística de las ventas por producto mediante:

- Valor mínimo
- Primer cuartil (Q1)
- Mediana
- Tercer cuartil (Q3)
- Valor máximo
- Promedio

El gráfico permite identificar variabilidad, concentración de datos y diferencias entre productos.

Además, se implementó una tabla estadística asociada al gráfico donde se muestran los valores calculados para cada producto:

Variable	Descripción
Mínimo	Menor cantidad de ventas registrada
Máximo	Mayor cantidad de ventas registrada
Promedio	Media de ventas del periodo analizado

La gráfica incorpora interacción mediante ventanas emergentes que muestran los valores estadísticos al posicionar el cursor sobre cada elemento visual.

Implementación de indicadores principales (KPIs)

Para complementar las visualizaciones se desarrollaron tarjetas informativas que muestran indicadores generales del comportamiento comercial:

- Ventas Totales
- Ingresos Totales
- Producto Líder
- Precio Promedio

Estos indicadores se calculan dinámicamente mediante JavaScript a partir de los datos filtrados, permitiendo que los valores cambien automáticamente conforme el usuario modifica los criterios de consulta.

Integración de filtros interactivos

Se incorporaron controles dinámicos para permitir la exploración personalizada de la información mediante filtros por:

- Producto
- Año
- Mes
- Periodo trimestral

El sistema permite realizar comparaciones temporales y analizar diferentes escenarios mediante agrupaciones de datos.

También se implementó un mecanismo de actualización dinámica:

- Al iniciar la aplicación se carga el histórico disponible hasta febrero de 2026.
- Mediante el botón "Actualizar Datos" se amplía la consulta hasta junio de 2026.

Esto permite simular un escenario real donde los datos pueden actualizarse conforme se incorporan nuevos registros.

Organización visual del dashboard

Para mejorar la interpretación de resultados, las visualizaciones fueron organizadas en bloques relacionados:

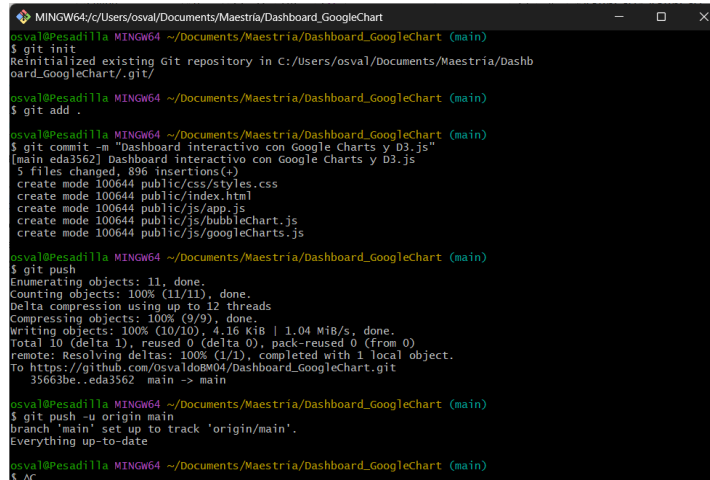
- Gráfico histórico acompañado de tabla resumen.
- Gráfico circular acompañado de descripción interpretativa.
- Bubble Chart acompañado de explicación de variables.
- Boxplot acompañado de tabla estadística.

Esta estructura permite que cada representación gráfica tenga un contexto asociado, facilitando la lectura de resultados y evitando que el usuario tenga que interpretar los gráficos de manera aislada.

Con estas implementaciones el dashboard pasó de ser una representación estática de datos a una herramienta interactiva de análisis, capaz de integrar procesamiento, filtrado, cálculo de indicadores y visualización avanzada en un entorno web.

8. Control de versiones con GitHub

Finalmente el proyecto fue almacenado utilizando Git: git init. Se agregaron los archivos: git add. Se creó un commit: git commit -m "Dashboard interactivo con Google Charts y D3.js". Y se sincronizó con GitHub: git push.



```
MINGW64/c/Users/osval/Documents/Maestria/Dashboard_GoogleChart
osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git init
Reinitialized existing Git repository in C:/Users/osval/Documents/Maestria/Dashb
oard_GoogleChart/.git/
osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git add .
osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git commit -m "Dashboard interactivo con Google Charts y D3.js"
[main eda3562] Dashboard interactivo con Google Charts y D3.js
5 files changed, 896 insertions(+)
 create mode 100644 public/css/styles.css
 create mode 100644 public/index.html
 create mode 100644 public/js/app.js
 create mode 100644 public/js/bubbleChart.js
 create mode 100644 public/js/googleCharts.js
osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git push
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 12 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (10/10), 4.16 KiB | 1.04 MiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
to https://github.com/OsvaldoBM04/Dashboard_GoogleChart.git
35663be..eda3562 main -> main
osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ git push -u origin main
branch 'main' set up to track 'origin/main'.
Everything up-to-date
osval@Pesadilla MINGW64 ~/Documents/Maestria/Dashboard_GoogleChart (main)
$ AC
```

El repositorio contiene:

- código fuente
- estructura del proyecto
- archivos JSON
- configuración Node.js

permitiendo reproducir la aplicación en otro equipo.

Conclusión

La aplicación desarrollada implementa un dashboard interactivo basado en una arquitectura cliente-servidor. Los datos se mantienen en un archivo JSON externo al código de visualización, son entregados mediante un servidor HTTP desarrollado con Node.js y Express, y posteriormente procesados mediante Google Charts y D3.js para generar visualizaciones dinámicas e interactivas.

Anexo:

<https://dashboard-googlechart.onrender.com/>

https://github.com/OsvaldoBM04/Dashboard_GoogleChart